# Guía de Snippets

**Go → Neovim** (blink.cmp) y **Vim** (mismas teclas que C++) · **C/C++ → Vim** (c-support) · + tutorial para crear los tuyos.

## 1. Snippets de Go (Neovim)

Los carga `~/.config/nvim/snippets/go.json` (blink.cmp los detecta solo). Escribís el **prefix** y confirmás con `Tab` / `Enter`. El buscador es difuso: alcanzan 2–3 letras.

| Prefix | Qué inserta | Descripción |
|---|---|---|
| `gmain` | `func main() { $0 }` | Función principal del programa |
| `gfunc` | `func ${1:name}() { $0 }` | Declaración de función genérica |
| `gerr` | `if err != nil { return err }` | Chequeo de error estándar |
| `gfor` | `for i, v := range ${1:items} { $0 }` | Bucle for range |
| `gfori` | `for i := 0; i < ${1:n}; i++ { $0 }` | For clásico de 3 partes |
| `gif` | `if ${1:cond} { $0 }` | If simple |
| `gifelse` | `if ${1:cond} { … } else { … }` | If / else |
| `gelse` | `else { $0 }` | Bloque else |
| `gwhile` | `for ${1:cond} { $0 }` | For con condición (while) |
| `gswitch` | `switch ${1:x} { case ${2:val}: … }` | Sentencia switch |
| `gcase` | `case ${1:val}:` | Case dentro de switch |
| `gprintf` | `fmt.Printf("${1:%v}\n", ${2:...})` | Equivalente a printf de C |
| `gscanf` | `fmt.Scan(&${1:val})` | Equivalente a scanf de C |
| `gstruct` | `type ${1:Name} struct { ${2:Field} ${3:Type} }` | Declaración de struct |
| `ginterface` | `type ${1:Name} interface { ${2:Method}() }` | Declaración de interfaz |
| `ggo` | `go func() { $0 }()` | Goroutine anónima |
| `gdefer` | `defer $0` | Sentencia defer |
| `ghandler` | `http.HandleFunc("${1:/}", func(w http.ResponseWriter, r *http.Request) { $0 })` | Handler HTTP (HTML) |
| `gserver` | `http.ListenAndServe("${1::8080}", nil)` | Levantar servidor HTTP |
| `gchan` | `ch := make(chan ${1:Type}, ${2:10})` | Declaración de channel |
| `gmutex` | `var mu sync.Mutex` | Mutex para sincronización |
| `gtest` | `func Test${1:Name}(t *testing.T) { $0 }` | Función de test |
| `gbench` | `func Benchmark${1:Name}(b *testing.B) { $0 }` | Función de benchmark |
| `gjson` | `json.Marshal(${1:v})` | Serializar a JSON |

**¿Cómo se usan los tabstops ($1, $2, $0)?** Al confirmar el snippet, el cursor queda parado en **$1** (primer campo, ya seleccionado). Escribís y con **Tab** saltás al siguiente (**$2**, **$3**...). **$0** es el final: quedás escribiendo normal. Ej. con `gstruct` : escribís `User` → Tab → `Name string` → Tab → `string` → Tab → listo.

## 1b. Snippets de Go en Vim — mismas teclas que C++

En `.go`, los atajos de c-support de C++ funcionan igual, traducidos a Go (`~/.vim/plugin/go_support.vim`). Tecla \:

### Condicionales (igual que C++)

| Tecla | Qué inserta |
|---|---|
| `\sif` | `if { … }` (cursor en la condición) |
| `\sie` | `if { } else { }` |
| `\sei` | `} else if { … }` |
| `\sel` | `else { … }` |
| `\sw` | `for { … }` (while) |
| `\ss` | `switch { … }` |
| `\sc` | `case` |
| `\sb` | `{ … }` (bloque) |
| `\sfo` | `for i := 0; i < n; i++ { … }` |
| `\sfr` | `for i, v := range items { … }` |

### Idioms (igual que C++)

| Tecla | Qué inserta |
|---|---|
| `\im` | `func main() { … }` |
| `\if` | `func name() { … }` |
| `\is` | `type Name struct { … }` |
| `\ipr` | `fmt.Printf("%v\n", …)` |
| `\isc` | `fmt.Scan(&val)` |

### Extras propios de Go

| Tecla | Qué inserta |
|---|---|
| `\se` | `if err != nil { return err }` |
| `\sm` / `\sf` | func main() / func name() (alias) |
| `\sr` / `\so` | for range / for clásico (alias) |
| `\sg` | `go func() { … }()` (en visual: envuelve la selección) |
| `\sd` | `defer …` |
| `\si` | `type Name interface { … }` |
| `\sh` | `http.HandleFunc("/", …)` |
| `\shs` | `http.ListenAndServe(":8080", nil)` |
| `\schan` | `ch := make(chan Type, 10)` |
| `\smu` | `var mu sync.Mutex` |
| `\stt` | `func TestName(t *testing.T) { … }` |
| `\sbm` | `func BenchmarkName(b *testing.B) { … }` |
| `\sj` | `json.Marshal(v)` |

**Visual:** seleccioná líneas y usá `\sif`, `\sw`, `\ss` o `\sg` para envolverlas en la estructura (if / for / switch / goroutine).

## 2. Snippets de C/C++ en Vim (c-support)

Tecla \ (misma que Go). Estructura: `\PALABRA`. Trabaja en modos normal **(n)**, insert **(i)** y visual **(v)**. Tabla completa extraída de `c-hotkeys.pdf` del propio plugin.

| Tecla | Qué hace |
|---|---|
| **Comentarios** | |
| `\cl` | Comentario fin de línea |
| `\cj` | Ajustar comentario fin de línea |
| `\cs` | Fijar columna del comentario fin de línea |
| `\c*` | Código ⇒ comentario `/* */` |
| `\cc` | Código ⇒ comentario `//` |
| `\co` | Comentario ⇒ código |
| `\cn` | Alternar comentario no-C |
| `\cfr` | Comentario de marco (frame) |
| `\cfu` | Comentario de función |
| `\cme` | Descripción de método |
| `\ccl` | Descripción de clase |
| `\cfdi` | Descripción de archivo (implementación) |
| `\cfdh` | Descripción de archivo (header) |
| `\ccs` | Secciones de archivo C/C++ (tab compl.) |
| `\chs` | Secciones de archivo H (tab compl.) |
| `\ckc` | Comentario keyword (tab compl.) |
| `\csc` | Comentario especial (tab compl.) |
| `\cma` | Macros de plantilla (tab compl.) |
| `\cd` | Fecha |
| `\ct` | Fecha y hora |
| `\cx` | Cambiar estilo de comentario C ↔ C++ |
| **Sentencias (control de flujo)** | |
| `\sfo` | `for ( ; ; )` |
| `\sfr` | `for ( : )` (range-based) |
| `\sd` | `do { } while (…)` |
| `\sw` | `while (…) { }` |
| `\sif` | `if (…) { }` |
| `\sie` | `if (…) { } else { }` |
| `\sei` | `else if (…) { }` |
| `\sel` | `else { }` |
| `\ss` | `switch (…) { }` |
| `\sc` | `case …:` |
| `\sb` | `{ }` (bloque) |
| **Idiomas (patrones comunes)** | |
| `\if` | Función |
| `\isf` | Función estática |
| `\im` | `main()` |
| `\ie` | `enum` + `typedef` |
| `\is` | `struct` + `typedef` |
| `\iu` | `union` + `typedef` |
| `\ipr` | `printf()` |
| `\isc` | `scanf()` |
| `\ica` | `p = calloc(...)` |
| `\ima` | `p = malloc(...)` |
| `\ire` | `p = realloc(...)` |
| `\isi` | `sizeof(...)` |
| `\ias` | `assert(...)` |

| | |
|---|---|
| `\ii` | Abrir archivo de entrada |
| `\io` | Abrir archivo de salida |
| `\ifsc` | `fscanf(...)` |
| `\ifpr` | `fprintf(...)` |
| `\i0` | `for( x=0; x<n; x+=1 )` |
| `\in` | `for( x=n-1; x>=0; x-=1 )` |

| Tecla | Qué hace |
|---|---|
| **# Preprocesador** | |
| `\pih` | Include header de la librería estándar |
| `\piph` | Include header POSIX |
| `\pg` | `#include <...>` (global) |
| `\pl` | `#include "..."` (local) |
| `\pd` | `#define` |
| `\pu` | `#undef` |
| `\pif` | `#if … #endif` |
| `\pie` | `#if … #else … #endif` |
| `\pid` | `#ifdef … #else … #endif` |
| `\pin` | `#ifndef … #else … #endif` |
| `\pind` | `#ifndef … #define … #endif` |
| `\pe` | `#error …` |
| `\pli` | `#line …` |
| `\pp` | `#pragma …` |
| `\pw` | `#warning …` |
| `\pi0` | `#if 0 … #endif` |
| `\pr0` | Quitar `#if 0 … #endif` |
| **Snippet (snippets propios)** | |
| `\nr` | Leer código snippet |
| `\nv` | Ver código snippet |
| `\nw` | Guardar código snippet |
| `\ne` | Editar código snippet |
| `\nf` / `\np` | Tomar prototipo de función |
| `\nm` | Tomar prototipo de método |
| `\ni` | Insertar prototipo(s) |
| `\nc` | Limpiar prototipo(s) |
| `\ns` | Mostrar prototipo(s) |
| `\ntl` | Editar plantillas locales |
| `\ntc` | Editar plantillas personalizadas |
| `\ntp` | Editar plantillas personales |
| `\ntr` | Recargar plantillas |
| `\ntw` | Asistente de configuración de plantillas |
| `\nts` | Elegir estilo de plantilla |
| `\njt` | Insertar jump tag |

### C++ (tecla `\` + `+`...)

| Tecla | Qué hace |
|---|---|
| `\+ih` | `#include` header estándar C++ |
| `\+ich` | `#include` header estándar C |
| `\+om` | Output manipulators |
| `\+fb` | ios flagbits |
| `\+c` | Clase |
| `\+cn` | Clase (usando `new`) |
| `\+tc` | Template class |
| `\+tcn` | Template class (usando `new`) |
| `\+ec` | Error class |
| `\+tf` | Template function |
| `\+tr` | `try … catch` |
| `\+ca` | `catch (…) { }` |
| `\+caa` | `catch(...) { }` |
| `\+ex` | `extern "C" { }` |
| `\+oif` | Abrir archivo de entrada |
| `\+oof` | Abrir archivo de salida |
| `\+uns` | `using namespace std;` |
| `\+un` | `using namespace xxx;` |
| `\+unb` | `namespace xxx { }` |
| `\+na` | Alias de namespace |
| `\+rt` | RTTI |
| `\+ic` | Implementación de clase |
| `\+icn` | Implementación de clase (con `new`) |
| `\+im` | Implementación de método |
| `\+ia` | Implementación de accessor |
| `\+itc` | Implementación de template class |
| `\+itcn` | Impl. template class (con `new`) |
| `\+itm` | Implementación de template method |
| `\+ita` | Implementación de template accessor |
| `\+ioi` | `operator >>` |
| `\+ioo` | `operator <<` |

### ▶ Ejecutar / Compilar / Make

| Tecla | Qué hace |
|---|---|
| `\rc` | Guardar y compilar |
| `\rl` | Linkear |
| `\rr` | Ejecutar el programa (en Vim sale en split abajo, buffer) |
| `\rt` | **Compilar y ejecutar en un terminal interactivo** (pestaña nueva; soporta `scanf` / `cin`) — mapeo agregado por nosotros |
| `\ra` | Argumentos de línea de comandos |
| `\rd` | Iniciar debugger |
| `\re` | Ejecutable a correr |
| `\rp` | Correr `splint` |
| `\rcc` | Correr `cppcheck` |
| `\rk` | Correr CodeCheck |
| `\ri` | Correr `indent` |
| `\rs` | Mostrar settings del plugin |
| `\ro` | Cambiar destino de salida (buffer/window) |
| `\rh` | Hardcopy del buffer |
| `\rm` | Correr `make` |
| `\rmc` | Correr `make clean` |

| | |
|---|---|
| `\rmd` | Correr `make doc` |
| `\rcm` | Elegir un makefile |
| `\rma` | Argumentos para `make` |

**Recordá:** el compilador y ejecución de C/C++ en Vim usan `\rt` (terminal interactivo — el que agregamos) y `\rr` (salida en split). Los IDs y comentarios de cabecera se configuran con `:NTW`.

## 🧩 3. Cómo crear tus propios snippets

### 3.1 En Neovim (archivo `~/.config/nvim/snippets/go.json`)

Es un JSON con formato VSCode: cada snippet es un objeto con `prefix`, `body` y `description`.

```
{
  "Go: hola": {
    "prefix": "ghola",
    "body": [
      "fmt.Println(\"${1:mensaje}\")",
      "// ${2:un comentario}",
      "$0"
    ],
    "description": "Saludar con fmt.Println"
  }
}
```

- `prefix` → las letras que escribís para que aparezca (¡podés acortar los que ya existan, ej. `gerr` → `iferr`!).
- `body` → array de líneas. `\t` = tab, `${1:valor}` = campo que saltás con Tab (el texto tras `:` es el valor inicial), `$0` = posición final.
- Guardás, y en cualquier `.go` ya aparece al escribir el prefix. No hay que reiniciar Neovim.

### 3.2 En Vim (archivos `~/.vim/plugin/go_support.vim` + `~/.vim/ftplugin/go.vim`)

Cada snippet es una entrada en el diccionario `s:go`. Agregá una:

```
let s:go.hola = {
     \ 'lines' : ["fmt.Println(\"hola\")", "\t// comentario"],
     \ 'cursor': [1, 5],    " fila y columna donde queda el cursor (0-based)
     \ 'mode'  : 'i' }      " 'i' = insertar, 'n' = normal
```

Y el atajo en `ftplugin/go.vim`, dentro del diccionario `s:go_maps`:

```
\ 'hola': ['hola'],    " ahora funciona con \shola
```

- `lines` → líneas que inserta (una línea vacía `""` = línea en blanco).
- `cursor` → fila/columna (0-based) donde queda el cursor. `-1` = primer carácter no blanco, `-2` = fin de la línea.
- TAB en Go es un carácter literal de tab (no espacios) — usá la tecla Tab real al escribir el archivo.

### 3.3 Recordatorio de rutas

| Editor | Archivo | Snippet que editás |
|---|---|---|
| Neovim | `~/.config/nvim/snippets/go.json` | Go (prefix `g…`) |
| Vim | `~/.vim/plugin/go_support.vim` | Definición de snippets Go `\s…` |
| Vim | `~/.vim/ftplugin/go.vim` | Mapeos `\s…` |
| Vim | `~/.vim/c-support/codesnippets/` | Snippets propios de c-support (C/C++) |
| Vim | `~/.vimrc.local` | Overrides locales (ej. `\rt`) |

Generado el día de hoy en la Raspberry Pi 400 · Los archivos fuente están en `~/Desktop/vim/` (carpetas `vimconfig/` y `lazyconfig/`).